HARD Mock Set 4 — Accenture Technical (Above-Level)

Level: Harder than the 10 uploaded sets. Unique questions — nothing here repeats Sets 1, 2, 3 or 5.

Format: 50 questions · Time: 50 minutes, closed book · No negative marking.

Section	Area	Qs
A	MS Office & Common Applications	8
B	Networking, Security & Cloud	12
C	CS Core — DSA / DBMS / OOP / HTML-CSS-JS	12
D	Pseudocode & Code Tracing	14
E	Applied — SQL & Code Output	4

Section A — MS Office & Common Applications (8)

Q1. A2:A6 = HR, IT, IT, FIN, IT and B2:B6 = 10, 20, 30, 40, 50. What does `=INDEX(B2:B6, MATCH("IT", A2:A6, 0))` return?

A) 30

B) 20

C) 40

D) `#N/A`

Q2. Which Excel function is **volatile** (recalculates every time the sheet recalculates)?

A) `SUM`

B) `VLOOKUP`

C) `IF`

D) `TODAY()`

Q3. A1 = "a", A2 = "" (empty), A3 = "c". What does `=TEXTJOIN("-", TRUE, A1:A3)` return?

A) `a-c`

B) `a--c`

C) `a-c-`

D) `#VALUE!`

Q4. You want row 1 and column A to stay visible while scrolling a large sheet. What do you do?

A) Freeze row 1, then freeze column A separately

B) Select A1 → View → Freeze Panes

C) Select **B2** → View → Freeze Panes

D) Split the window and hide row 1

Q5. You want each reviewer's insertions and deletions to appear in their own colour with their name attached. Which feature?

A) Comments

B) Track Changes

C) Mail Merge

D) Document Inspector

Q6. You paste slides from another deck and they arrive with the *source* deck's fonts and colours. How do you force them to adopt the **current** deck's design immediately after pasting?

A) Re-apply the theme from the Design tab

B) Right-click the slide and choose Reset Slide

C) Nothing — pasted slides always keep their source formatting

D) Use the Paste Options button → **Use Destination Theme**

Q7. Approximately how many **megabytes per second** can a 1 Gbps link transfer?

A) 1000 MB/s

B) 8 MB/s

C) 125 MB/s

D) 1024 MB/s

Q8. Which Windows shortcut **locks** the computer?

A) Win + L

B) Ctrl + Alt + Del only

C) Win + K

D) Alt + F4

Section B — Networking, Security & Cloud (12)

Q9. What is the **network address** of `172.20.55.130/25`?

A) 172.20.55.0

B) 172.20.55.64

C) 172.20.0.0

D) 172.20.55.128

Q10. Which **wildcard mask** matches a `/26` subnet mask (used in ACLs/OSPF network statements)?

A) 0.0.0.31

B) 0.0.0.63

C) 0.0.0.127

D) 255.255.255.192

Q11. Which port is used for **secure mail submission** from a client to an SMTP server?

A) 25

B) 110

C) 587

D) 143

Q12. A host must send a packet to an IP address **outside its own subnet**. Which MAC address does it ARP for?

A) The MAC address of its **default gateway**

B) The MAC address of the destination host

C) The MAC address of the DNS server

D) `FF:FF:FF:FF:FF:FF` only

Q13. Why does a broadcast (e.g. an ARP request) **not** cross a router?

A) Routers run at Layer 2 and block all broadcasts by MAC filtering

B) Broadcasts are converted to unicast automatically

C) The TTL of a broadcast frame is 1

D) A router terminates the **broadcast domain** — it does not forward broadcast traffic

Q14. Which technology gives default-gateway redundancy to hosts **without** changing anything on the hosts?

A) STP

B) HSRP / VRRP (first-hop redundancy with a virtual IP)

C) DHCP failover

D) Port-channel

Q15. Which DNS record marks the **start of authority** for a zone (primary name server, refresh timers)?

A) NS

B) A

C) SOA

D) TXT

Q16. Which DDoS technique uses **open recursive resolvers** and a spoofed source IP to multiply traffic?

A) DNS amplification / reflection

B) SYN flood

C) Slowloris

D) ARP cache overflow

Q17. Which cloud storage type is best for serving images and videos directly over HTTP at scale?

A) Block storage

B) File/NAS storage

C) Archival tape

D) Object storage (e.g. S3, Azure Blob)

Q18. Why are passwords stored as **salted** hashes rather than plain hashes?

A) Salting makes the hash shorter

B) A unique salt defeats rainbow-table attacks and makes identical passwords hash differently

C) Salting encrypts the password

D) Salting is required by TCP

Q19. IPsec in **tunnel mode** encrypts:

A) Only the TCP/UDP payload

B) The IP header only

C) The entire original IP packet, which is then wrapped in a new IP header

D) Nothing — tunnel mode only authenticates

Q20. Which DNS-published mechanism tells receiving mail servers which hosts may send mail for a domain?

A) SPF (a TXT record)

B) HTTPS

C) DNSSEC

D) CAA

Section C — CS Core: DSA / DBMS / OOP / HTML-CSS-JS (12)

Q21. Complexity of `for i = 1..n: for j = 1..n: for k = 1..100 (O(1))`?

A) $O(n^3)$

B) O(n log n)

C) O(100n)

D) O(n^2)

Q22. Which collision-resolution method suffers from **primary clustering**?

A) Separate chaining

B) Linear probing

C) Double hashing

D) Quadratic probing with a prime table size

Q23. Which traversal visits tree nodes **level by level**?

A) Breadth-first (level-order), using a queue

B) Pre-order DFS

C) In-order DFS

D) Post-order DFS

Q24. With pushes in the order 1, 2, 3, 4, which **pop** sequence is **impossible**?

A) 4 3 2 1

B) 2 1 4 3

C) 3 1 2 4

D) 1 2 3 4

Q25. Which statement about **foreign keys** is TRUE?

A) A foreign key must always be the primary key of the child table

B) A foreign key column cannot contain NULL

C) A foreign key automatically creates an index on the parent table

D) A NULL is allowed in a nullable FK column, but any non-NULL value must exist in the parent table

Q26. `Orders(cust_id, product)` → (1,A), (1,A), (1,B), (2,A), (3,C). What is the **second row** of

```
SELECT cust_id, COUNT(DISTINCT product) FROM Orders GROUP BY cust_id ORDER BY cust_id;
```

?

A) (1, 3)

B) (2, 1)

C) (3, 1)

D) (2, 2)

Q27. Relation with composite primary key `(A, B)` and FD `A → C`. Which normal form is violated?

A) 2NF (partial dependency on part of the key)

B) 3NF only

C) 1NF

D) None — C depends on the whole key

Q28. T1 reads a row; T2 updates and commits it; T1 re-reads the row and sees a **different value**. Which anomaly?

A) Dirty read

B) Phantom read

C) Non-repeatable read

D) Lost update

Q29. In Java, which statement is TRUE?

A) A class can extend multiple classes

B) An interface can contain method bodies for all its methods

C) An abstract class cannot have constructors

D) A class can implement multiple interfaces but extend only one class

Q30. With `box-sizing: border-box; width: 200px; padding: 20px; border: 5px solid`, the **content box** is:

A) 200px

B) 150px

C) 120px

D) 190px

Q31. JavaScript:

```
function counter() { let n = 0; return () => ++n; }
const c = counter();
c(); c();
console.log(c());
```

A) 3

B) 1

C) 0

D) `undefined`

Q32. Binary `1010 1100` in decimal is:

A) 154

B) 204

C) 172

D) 188

Section D — Pseudocode & Code Tracing (14)

Q33.

```
n ← 50; c ← 0
WHILE n > 0
    IF n MOD 7 = 0
        BREAK
    END IF
    n ← n - 3
    c ← c + 1
END WHILE
PRINT c, n
```

A) 16 2

B) 4 38

C) 17 -1

D) 5 35

Q34.

```
FUNCTION F(n)
    IF n < 3
        RETURN n
    RETURN F(n-1) * F(n-3)
END FUNCTION
PRINT F(6)
```

A) 0

B) 6

C) 12

D) 18

Q35.

```
count ← 0
FOR i ← 1 TO 3
    FOR j ← 1 TO 3
        IF i + j > 4
            count ← count + 1
        END IF
    END FOR
END FOR
PRINT count
```

A) 6

B) 4

C) 3

D) 9

Q36.

```
n ← 120
rev ← 0; m ← n
WHILE m > 0
    rev ← rev * 10 + m MOD 10
    m ← m DIV 10
END WHILE
IF rev = n
    PRINT "PALINDROME"
ELSE
    PRINT "NOT"
END IF
```

A) PALINDROME (120 reversed is 120)

B) PALINDROME (leading zeros are preserved)

C) NOT (rev = 021 = 21, but 021 = 120 numerically)

D) NOT (rev = 21, which is not equal to 120)

Q37.

```
A ← [10, 20, 30, 40]
FOR i ← 3 DOWNTO 1
    A[i] ← A[i - 1]
END FOR
A[0] ← 5
PRINT A
```

A) [5, 20, 30, 40]

B) [5, 10, 20, 30]

C) [10, 20, 30, 5]

D) [5, 10, 10, 20]

Q38.

```
a ← 2; b ← 8
WHILE a < b
    a ← a + 3
    b ← b - 1
END WHILE
PRINT a, b
```

A) 8 6

B) 5 7

C) 11 5

D) 8 5

Q39.

```
calls ← 0
FUNCTION T(n)
    calls ← calls + 1
    IF n <= 1
        RETURN 1
    RETURN T(n-1) + T(n-1)
END FUNCTION
PRINT T(4), calls
```

A) 16 31

B) 8 8

C) 8 15

D) 4 15

Q40.

```
s ← ""
FOR i ← 1 TO 3
    FOR j ← 1 TO i
        s ← s + "*"
    END FOR
    s ← s + "-"
END FOR
PRINT LENGTH(s)
```

A) 6

B) 7

C) 8

D) 9

Q41. (g is a **global** variable; the parameter has the same name)

```
g ← 100
FUNCTION p(g)
    g ← g * 2
    RETURN g
END FUNCTION
PRINT p(5), g
```

A) 200 200

B) 10 100

C) 10 10

D) 200 100

Q42.

```
A ← [7, 3, 9, 3]
mi ← 0
FOR i ← 1 TO 3
    IF A[i] < A[mi]
        mi ← i
    END IF
END FOR
PRINT mi
```

A) 1

B) 3

C) 0

D) 2

Q43.

```
FUNCTION rev(s)
    IF LENGTH(s) <= 1
        RETURN s
    RETURN rev(SUBSTRING(s, 2)) + s[0]
END FUNCTION
PRINT rev("abcd")
```

A) `abcd`

B) `abdc`

C) `dcba`

D) `dbca`

Q44.

```
n ← 5; out ← 1
REPEAT
    out ← out * n
    n ← n - 1
UNTIL n = 0
PRINT out
```

A) 720

B) 24

C) 60

D) 120

Q45. Bitwise (integers are 8-bit here):

```
x ← 6        // 0000 0110
y ← 3        // 0000 0011
PRINT (x XOR y), (x << 1), (x AND y)
```

A) 5 6 2

B) 5 12 2

C) 7 12 3

D) 3 12 2

Q46. Sorted-merge two-pointer:

```
A ← [1, 4, 7]
B ← [2, 3, 9]
i ← 0; j ← 0; out ← empty list
WHILE i < 3 AND j < 3
    IF A[i] < B[j]
        APPEND A[i] TO out
        i ← i + 1
    ELSE
        APPEND B[j] TO out
        j ← j + 1
    END IF
END WHILE
PRINT out
```

A) [1, 2, 3, 4, 7, 9]

B) [1, 2, 3, 4, 9, 7]

C) [1, 2, 3, 4, 7]

D) [1, 4, 2, 3, 7, 9]

Section E — Applied: SQL & Code Output (4)

`Emp(id, dept, salary)` — 6 rows: (1, IT, 60k), (2, IT, 80k), (3, HR, 40k), (4, IT, 50k), (5, FIN, 90k), (6, HR, 55k)

Q47. Which query lists employees earning **more than their own department's average**?

A) `SELECT * FROM Emp WHERE salary > AVG(salary) GROUP BY dept;`

B) `SELECT * FROM Emp e WHERE salary > (SELECT AVG(salary) FROM Emp WHERE dept = e.dept);`

C) `SELECT * FROM Emp GROUP BY dept HAVING salary > AVG(salary);`

D) `SELECT * FROM Emp WHERE salary > (SELECT AVG(salary) FROM Emp ORDER BY dept);`

Q48. What does `SELECT COUNT(DISTINCT dept) FROM Emp` return?

A) 3

B) 6

C) 2

D) 4

Q49. Python:

```
s = "abc"
s[0] = "z"
```

A) `s` becomes `"zbc"` (strings are mutable)

B) `s` becomes `"abc"` with no error, silently ignored

C) Works only for single characters

D) `TypeError: 'str' object does not support item assignment`

Q50. JavaScript — the handler below works and changes the button text. What happens if you replace the regular function with an **arrow function** `() => { this.textContent = 'Added'; }`?

```
document.getElementById('btn').addEventListener('click', function () {
  this.textContent = 'Added';
});
```

A) Identical — `this` is always the element in an event handler

B) The text changes, but only on the second click

C) `this` is no longer the button (it is `undefined` in strict mode / the global object), so the button text does not change

D) A syntax error, because arrow functions cannot be used as event handlers

ANSWER KEY

Do not look here until you have finished all 50 questions.

— Answer Key for Set 4

Quick grid

Q	1	2	3	4	5	6	7	8	9	10
Ans	B	D	A	C	B	D	C	A	D	B

Q	11	12	13	14	15	16	17	18	19	20
Ans	C	A	D	B	C	A	D	B	C	A

Q	21	22	23	24	25	26	27	28	29	30
Ans	D	B	A	C	D	B	A	C	D	B

Q	31	32	33	34	35	36	37	38	39	40
Ans	A	C	D	A	C	D	B	A	C	D

Q	41	42	43	44	45	46	47	48	49	50
Ans	B	A	C	D	B	C	B	A	D	C

Explanations

Q1 — B. `MATCH("IT", A2:A6, 0)` returns **2** (first exact hit, A3). `INDEX(B2:B6, 2)` = the 2nd cell of that range = **20**. INDEX/MATCH beats VLOOKUP because the lookup column need not be leftmost.

Q2 — D. Volatile functions recalculate on every recalculation: `TODAY()`, `NOW()`, `RAND()`, `RANDBETWEEN()`, `OFFSET()`, `INDIRECT()`, `INFO()`, `CELL()`.

Q3 — A. `ignore_empty = TRUE` skips the blank, so the parts are "a" and "c" joined by "-" → **a-c**. With FALSE you would get `a--c`.

Q4 — C. Freeze Panes freezes everything **above and to the left of the selected cell** — select B2 to freeze row 1 and column A together.

Q5 — B. Track Changes records insertions/deletions/formatting per author, in the author's colour, with a name and timestamp. Comments only annotate.

Q6 — D. The Paste Options icon (Ctrl) offers **Use Destination Theme** (adopt current deck), **Keep Source Formatting**, or **Picture**.

Q7 — C. 1 Gbps = 1000 Mbps ÷ 8 = **125 MB/s** (decimal megabytes). At the binary MiB scale you would quote ≈ 119 MiB/s.

Q8 — A. Win + L locks the workstation. Ctrl + Alt + Del opens the security screen; Alt + F4 closes the active window.

Q9 — D. /25 = 255.255.255.128 → block 128 → 130 is in 128–255 → **172.20.55.128**.

Q10 — B. /26 = 255.255.255.192 → wildcard = 255.255.255.255 − mask = **0.0.0.63**.

Q11 — C. 587 = submission (STARTTLS); 25 = server-to-server relay; 465 = implicit TLS (legacy); 110 = POP3; 143 = IMAP.

Q12 — A. Off-subnet traffic always goes to the default gateway, so the host ARPs for the **gateway's MAC**; the destination IP stays in the IP header.

Q13 — D. Routers do not forward L2 broadcasts — each router interface ends a broadcast domain (collision domains end at switch ports).

Q14 — B. HSRP/VRRP share a **virtual IP + virtual MAC**; hosts keep one static gateway and failover is transparent.

Q15 — C. `SOA` = start of authority (primary NS, serial, refresh/retry/expire). `NS` delegates; `A`/`AAAA` map names; `TXT` carries arbitrary text.

Q16 — A. DNS amplification: small spoofed queries to open resolvers → large responses aimed at the victim (reflection + amplification).

Q17 — D. Object storage (S3, Blob) is flat, HTTP-native and horizontally scalable — ideal for media. Block storage = VM disks; file storage = shared NFS/SMB.

Q18 — B. A per-user random salt means precomputed rainbow tables are useless and two identical passwords produce different hashes.

Q19 — C. Tunnel mode encrypts the **whole original IP packet** and adds a new outer IP header (site-to-site VPN). Transport mode protects only the payload.

Q20 — A. SPF (TXT record) lists authorised sending hosts. DKIM signs messages; DMARC is the policy layer on top of both.

Q21 — D. The inner loop is a **constant** 100 iterations → O(n²) × O(1) = **O(n²)**.

Q22 — B. Linear probing creates long contiguous occupied runs (primary clustering). Chaining, double hashing and quadratic probing avoid it.

Q23 — A. Level-order = BFS with a queue. DFS variants (pre/in/post) use a stack/recursion.

Q24 — C. To pop 3 first you push 1, 2, 3 → the top is now 2, so you cannot pop **1** before 2. Sequence 3 1 2 4 is impossible.

Q25 — D. NULL in a nullable FK is fine (unknown/absent relationship); a non-NULL value must match a parent key (referential integrity).

Q26 — B. Rows: (1, 2 distinct products), **(2, 1)**, (3, 1) → second row = (2, 1).

Q27 — A. `A → C` depends on **part** of the composite key → partial dependency → violates **2NF**.

Q28 — C. Non-repeatable read: the same row read twice in one transaction returns different values because another transaction committed an update. A phantom is about *new rows* appearing.

Q29 — D. Java: single inheritance of implementation, multiple inheritance of *type* via interfaces (which may also have `default` methods).

Q30 — B. `border-box` = declared width includes padding and border → content = 200 – (2×20) – (2×5) = **150px**.

Q31 — A. The returned arrow function closes over the same `n`, which persists between calls → 1, 2, **3**.

Q32 — C. 128 + 32 + 8 + 4 = **172**.

Q33 — D. 50 → 47 → 44 → 41 → 38 → 35; 35 MOD 7 = 0 → BREAK after **5** decrements with `n = 35`.

Q34 — A. F(0)=0, F(1)=1, F(2)=2, F(3)=F(2)×F(0)=0 → every later value multiplies by F(3) or F(4) = 0 → **F(6) = 0**.

Q35 — C. Pairs with i + j > 4: (2,3), (3,2), (3,3) → **3**.

Q36 — D. Reversing digits produces `rev = 21` (the leading zero of "021" is lost as a number), and 21 ≠ 120 → **NOT**.

Q37 — B. Shift right from the end: [10,10,20,30] → [5,10,20,30]. Shifting **backwards** is essential; going forwards would copy 10 everywhere.

Q38 — A. (2,8) → (5,7) → (8,6); 8 < 6 is false → `8 6`.

Q39 — C. T(4) = 2³ = **8**. Calls: C(1)=1, C(n)=1+2C(n–1) → C(4) = **15**. Exponential duplication — memoise it!

Q40 — D. "*-" (2) + "**-" (3) + "***-" (4) = **9** characters.

Q41 — B. The parameter shadows the global inside `p`, so `g ← 10` locally; the global `g` is untouched → `10 100`.

Q42 — A. Min index: 7 → 3 at index 1; 9 no; the second 3 is **not** `< 3`, so the index stays **1** (strict comparison keeps the earliest minimum).

Q43 — C. `rev("abcd")` = rev("bcd") + "a" = … = **"dcba"**.

Q44 — D. REPEAT…UNTIL tests **after** the body, so n = 5,4,3,2,1 all multiply → 5! = **120** (and it would run once even if n were already 0).

Q45 — B. 6 XOR 3 = 5 (`0110 ^ 0011 = 0101`); 6 << 1 = 12; 6 AND 3 = 2 → `5 12 2`.

Q46 — C. Trace: 1, 2, 3, 4, 7 — then `i = 3` ends the loop and **9 is never appended**. Real merges need a clean-up step for the remaining tail.

Q47 — B. A **correlated** subquery re-evaluates `AVG(salary)` for each row's department. (A) and (C) put an aggregate in WHERE/HAVING illegally; (D) returns one scalar for the whole table.

Q48 — A. Distinct departments: IT, HR, FIN → **3**.

Q49 — D. Python strings are immutable — item assignment raises `TypeError`. Use `"z" + s[1:]`.

Q50 — C. Arrow functions do **not** bind their own `this`; they inherit it from the enclosing scope, so `this.textContent` is not the button. Use a regular function (or `event.target`) when you need the element.

Score guide (Set 4)

Score	Verdict
43–50	Excellent — safely above Accenture level
34–42	Good — revise the missed areas
25–33	Needs work — revisit the notes, retry after 3 days
< 25	Return to chapters 01–06 before Set 5